UNIVERSITY OF RWANDA

Gako Campus

Department of Computer Engineering

Year 4 · Semester I

Module: Real-Time Systems

Unit 2 Lab Report

Real-Time Scheduling Mini Projects 2.1 – 2.3

RM, EDF and Priority Inheritance Simulation · RM versus EDF Schedulability Tool · Partitioned Multiprocessor Scheduling

GROUP ASSIGNMENT

No.	Full name	Registration number
1	Janvier NSHIMIYIMANA	223026679
2	Blandon ASHIMIRWE	223026675
3	Gilbert HIRWA NTWALI	223026689

Source code repository

https://github.com/hirwagilbert/lab--2-real-time-system-

Submitted: October 2026

Table of Contents

1. Introduction **3**
1.1 Objectives 3
1.2 Environment and repository layout 3
2. Mini Project 2.1 – Simulating RM, EDF and Priority Inheritance **4**
2.1 Task model and utilization tests 4
2.2 RM and EDF simulation 4
2.3 Response Time Analysis of τ4 5
2.4 Priority inversion and the PIP fix 5
2.5 Why the simulated PIP blocking matches the bound 6
2.6 Lab tasks 7
3. Mini Project 2.2 – RM versus EDF Schedulability Comparison Tool **10**
3.1 Tool design 10
3.2 Results on seven task sets 10
3.3 Written report 11
3.4 Reflection: when the bound fails but RTA succeeds 13
4. Mini Project 2.3 – Partitioned Multiprocessor Scheduling Simulator **15**
4.1 Design 15
4.2 Test sets and comparison table 15
4.3 Per-core schedule 16
4.4 Written analysis 16
5. Conclusion **17**
References **18**
Appendix A – Reproducing the results **18**

1. Introduction

This report documents our group's solution to the three mini projects of the Unit 2 lab manual of the Real-Time Systems module. All three projects were implemented as pure-Python discrete-event (tick-based) simulators – no RTOS, board or C compiler is needed – and every number, table and chart in this report was produced by the code in our repository, so the results can be reproduced exactly by re-running the scripts listed in Appendix A.

1.1 Objectives

- **Mini Project 2.1** – build a simulator for Rate Monotonic (RM) and Earliest Deadline First (EDF), apply the Liu & Layland and EDF U ≤ 1 tests, run Response Time Analysis (RTA), and reproduce priority inversion together with its Priority Inheritance Protocol (PIP) fix.
- **Mini Project 2.2** – generalise the simulator into a tool that takes any periodic task set (from a list or a CSV file) and reports RM/DM and EDF schedulability with an RTA fallback and a two-panel comparison chart.
- **Mini Project 2.3** – partition task sets over m cores with First-Fit, First-Fit Decreasing, Best-Fit Decreasing and Worst-Fit, using the per-core Liu & Layland bound as the admission test.

1.2 Environment and repository layout

Python 3.10+ and matplotlib (charts drawn with broken_barh) are the only requirements; pandas was not needed. Time is discrete: one tick equals one time unit (1 ms in the unit's examples), all jobs of a task are released synchronously at multiples of T_i, and a job that misses its deadline is recorded and allowed to finish late. The complete code is available at https://github.com/hirwagilbert/lab--2-real-time-system-.

Table 1. *Repository contents.*

File / folder	Purpose
rts_sim.py	Shared core: Task model, utilization tests, RTA, RM/DM/EDF/LLF tick simulators, mutex simulator with/without PIP, Gantt helpers
mp21_run.py	Runs every procedure step and lab task of Mini Project 2.1
schedcheck.py	Mini Project 2.2 command-line tool (CSV or Python list input)
tasksets/*.csv	The seven task sets analysed in Mini Project 2.2
partition.py	Mini Project 2.3: FF, FFD, BFD, WF partitioning and per-core simulation
tests/test_all.py	Hand-computed checks (bounds, RTA, PIP blocking, verdicts, partitions)
figures/, results/	Generated PNG charts and the printed text output of each script
report/	This report (PDF) and the script that builds it

2. Mini Project 2.1 – Simulating RM, EDF and Priority Inheritance

2.1 Task model and utilization tests

Each periodic task τ_i is a Task dataclass holding name, C_i, T_i and D_i (D_i defaults to T_i, the implicit-deadline model of Section 2.1.2) with the derived utilization $u_i = C_i/T_i$. The functions total_utilization(tasks) and rm_bound(n) = n(2^(1/n) − 1) implement Section 2.2.1, and the EDF test checks U ≤ 1 (Section 2.3.1). Note that the lab manual's printed formula "2 * (1/n)" is a typesetting slip for the power 2^(1/n); our implementation uses the power, which is what gives the expected value 0.780 for n = 3.

Table 2. Utilization tests reproduced for the unit's two worked examples.

Example	Tasks (C, T)	U	L&L bound	RM bound test	EDF U ≤ 1
Section 2.2.2 (3 tasks)	τ1(1,4) τ2(2,6) τ3(2,12)	0.750	0.780	PASS	PASS
Section 2.3.2 (+ τ4: C=1, T=8)	τ1(1,4) τ2(2,6) τ3(2,12) τ4(1,8)	0.875	0.757	FAIL (inconclusive)	PASS

Both results match the unit exactly: the three-task set (U = 0.750 ≤ 0.780) is guaranteed RM-schedulable by the sufficient test, while adding τ4 raises U to 0.875, above rm_bound(4) = 0.757. The RM test is then **inconclusive** (it is only sufficient), whereas EDF's exact test U ≤ 1 still guarantees schedulability.

2.2 RM and EDF simulation

simulate_rm(tasks, sim_time) steps through time one tick at a time. At every tick t it releases a new job of τ_i whenever t is a multiple of T_i, records any job whose absolute deadline has passed unfinished, and runs the ready job of the highest-priority task (shortest period), preempting the running job if necessary. simulate_edf is identical except that the selection key is the job's absolute deadline; ties on deadline are broken in favour of the task listed first (the shorter-period task). Both return the per-tick timeline, deadline misses, response times and context-switch counts. Figure 1 shows the four-task set over one hyperperiod (LCM(4, 6, 12, 8) = 24 ms).

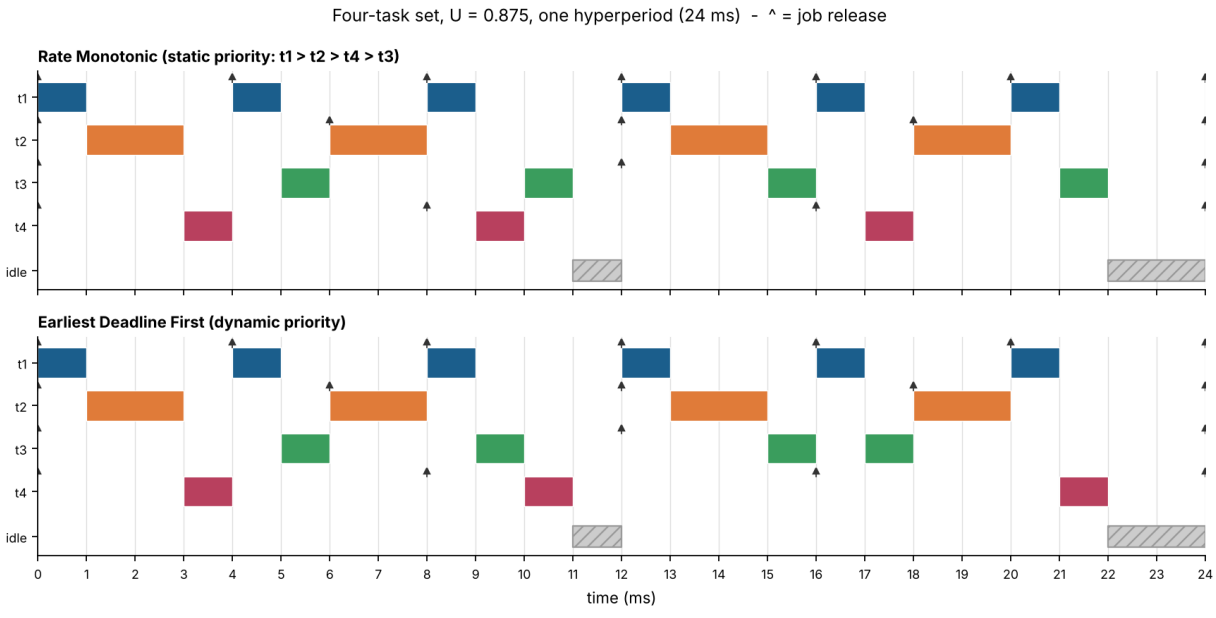


Figure 1. RM (top) and EDF (bottom) schedules of the four-task set over one hyperperiod. Triangles mark job releases; grey hatching is idle time.

Table 3. Simulated metrics over one hyperperiod (24 ms).

Policy	Deadline misses	Context switches	Worst R(τ1)	Worst R(τ2)	Worst R(τ3)	Worst R(τ4)	Idle
RM	0	15	1	3	11	4	3
EDF	0	15	1	3	10	6	3

Observations. τ1 executes in exactly the same slots in both charts, because it always has both the shortest period and the earliest deadline. The schedules diverge at t = 9, 10, 17 and 21, where the jobs of τ3 and τ4 interleave: RM always favours τ4 (period 8) over τ3 (period 12), whereas EDF runs whichever job has the nearer absolute deadline – at t = 9 the first job of τ3 (deadline 12) is served before the second job of τ4 (deadline 16). EDF therefore lowers τ3's worst-case response time from 11 to 10 ms at the cost of τ4 (4 → 6 ms), i.e. it spreads the slack according to urgency instead of according to a static rate. Total busy time is identical (21 of 24 ms, U = 0.875) because both algorithms are work-conserving, and both meet every deadline – something the RM utilization bound alone could not promise.

2.3 Response Time Analysis of τ4

Because the utilization bound was inconclusive, we applied the exact RTA recurrence of Section 2.5.2, $R_i = C_i + \sum_{j \in hp(i)} \lceil R_i/T_j \rceil \cdot C_j$, iterating from $R_i = C_i$ until convergence or until R_i exceeds D_i. Under RM the priority order is τ1 (T=4) > τ2 (T=6) > τ4 (T=8) > τ3 (T=12); τ4 is therefore third, and τ3 – not τ4 – is the lowest-priority task, so we analysed both.

```
τ4: hp = {τ1, τ2}       R⁰ = 1
     R¹ = 1 + ⌈1/4⌉·1 + ⌈1/6⌉·2 = 4
     R² = 1 + ⌈4/4⌉·1 + ⌈4/6⌉·2 = 4   → converged, R4 = 4 ≤ D4 = 8   ✔
τ3: hp = {τ1, τ2, τ4}   R: 2 → 6 → 7 → 9 → 11 → 11  → R3 = 11 ≤ D3 = 12  ✔
```

Table 4. RTA results under RM for the four-task set.

Task	RM priority	C	T = D	R (RTA)	Iterations	Verdict
τ1	1	1	4	1	1	OK
τ2	2	2	6	3	2	OK
τ4	3	1	8	4	2	OK
τ3	4	2	12	11	5	OK

RTA gives a definitive answer where the utilization bound could not: R4 = 4 ≤ 8 and every $R_i \le D_i$, so the set **is RM-schedulable**. The analytical values agree exactly with the worst response times observed in the simulation (Table 3), which validates both the simulator and the RTA implementation.

2.4 Priority inversion and the PIP fix

For Section 2.6.1 we built a second simulator, simulate_resources, in which each job is a list of segments – normal execution or a critical section on a named mutex. The scenario is: low-priority L takes mutex S at t = 0 and needs 6 ticks inside it; high-priority H arrives at t = 1 and needs S for 2 ticks; medium-priority M arrives at t = 2 and computes for 4 ticks without touching S. With protocol = 'none' the scheduler simply runs the highest-priority job that is not blocked. With protocol = 'pip', every tick the holder of a mutex inherits the highest priority among the jobs blocked on it (applied transitively);

because the base priority is never overwritten, it is automatically restored as soon as the mutex is released.

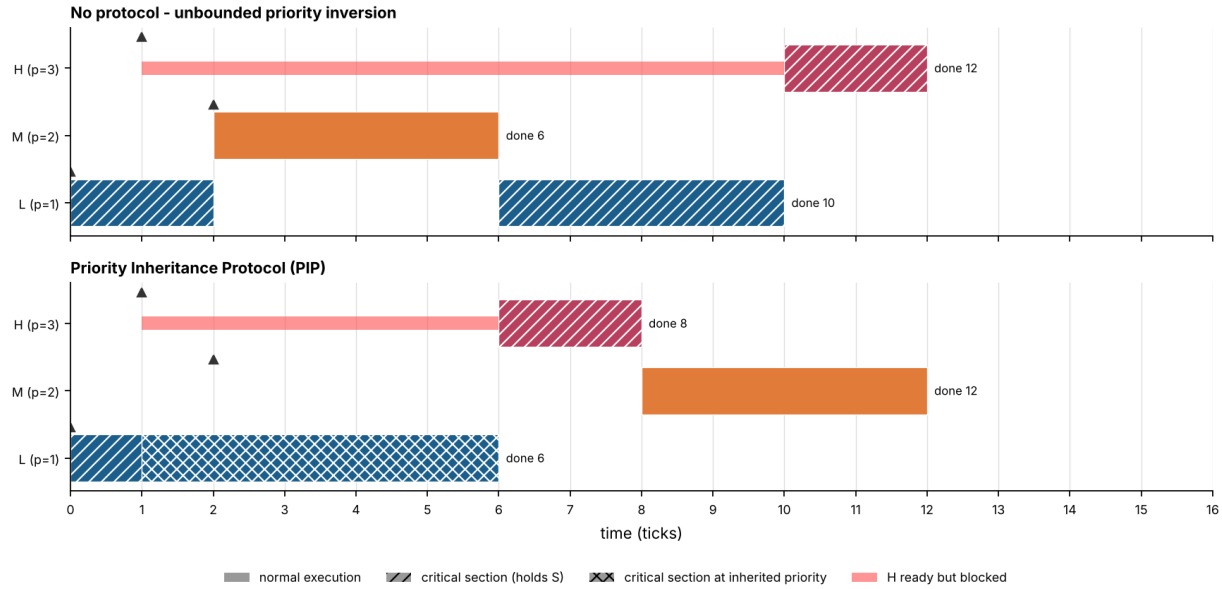


Figure 2. Priority inversion without a protocol (top) and with the Priority Inheritance Protocol (bottom). Hatched = inside the critical section; cross-hatched = L running at H's inherited priority; red band = H ready but blocked.

Without a protocol, M preempts L at t = 2 even though H is waiting for L, so H is delayed by the rest of L's critical section **and** by all of M's execution; it completes at t = 12. With PIP, L runs at H's priority from t = 1, M cannot preempt it, L releases S at t = 6, H completes at t = 8 and M runs afterwards. This is the simulated analogue of the Mars Pathfinder fix.

Table 5. Measured blocking of H (arrival t = 1, C_H = 2).

Scenario	Protocol	H finishes	Response R_H	H blocked (ticks)	PIP bound
L, M, H	None	12	11	9 = 5 (CS of L) + 4 (M)	—
L, M, H	PIP	8	7	5	≤ 6
L, M, M2, H	None	15	14	12 = 5 + 4 + 3	—
L, M, M2, H	PIP	8	7	5	≤ 6

2.5 Why the simulated PIP blocking matches the bound

Section 2.7.1 states that under the Priority Inheritance Protocol a high-priority job can be blocked for at most one critical section of each lower-priority job that shares a resource with it. In our scenario only one lower-priority job, L, shares the mutex S with H, and L's critical section lasts 6 ticks, so the bound on H's blocking is 6 ticks. The simulator measured 5 ticks, which respects the bound, and the difference is fully explained by the timeline: L had already executed one tick of its critical section before H arrived at t = 1, so only the remaining 5 ticks could block H. The decisive point is what happens at t = 2. Without a protocol, M has a higher priority than L, preempts it, and H must additionally wait for all 4 ticks of M, an unrelated task; the blocking grows to 9 ticks and, when we added a second medium task M2, to 12 ticks. Inversion of this kind is unbounded because it depends on how much medium-priority work happens to arrive. Under PIP, L inherits H's priority the moment H blocks, so neither M nor M2 can preempt it, and as soon as L releases S its base priority is restored and H runs immediately. H's blocking

is therefore limited to the remainder of one critical section of L – 5 ticks with one or two medium tasks alike – exactly as the one-critical-section-per-lower-priority-task bound predicts.

2.6 Lab tasks

Lab task 1 – Deadline Monotonic

simulate_dm orders priorities by relative deadline. On the implicit-deadline four-task set its timeline is identical, tick for tick, to RM ($D_i = T_i$ makes the two orders the same). We then shortened τ3's deadline to D3 = 3 (τ1 (1,4,4), τ2 (2,6,6), τ3 (2,12,3)). RM still gives τ3 the lowest priority and it misses its first deadline (RTA: R3 = 5 > 3), whereas DM promotes τ3 to the top and all tasks meet their deadlines (R3 = 2, R1 = 3, R2 = 6). The schedule changes only when some $D_i \neq T_i$.

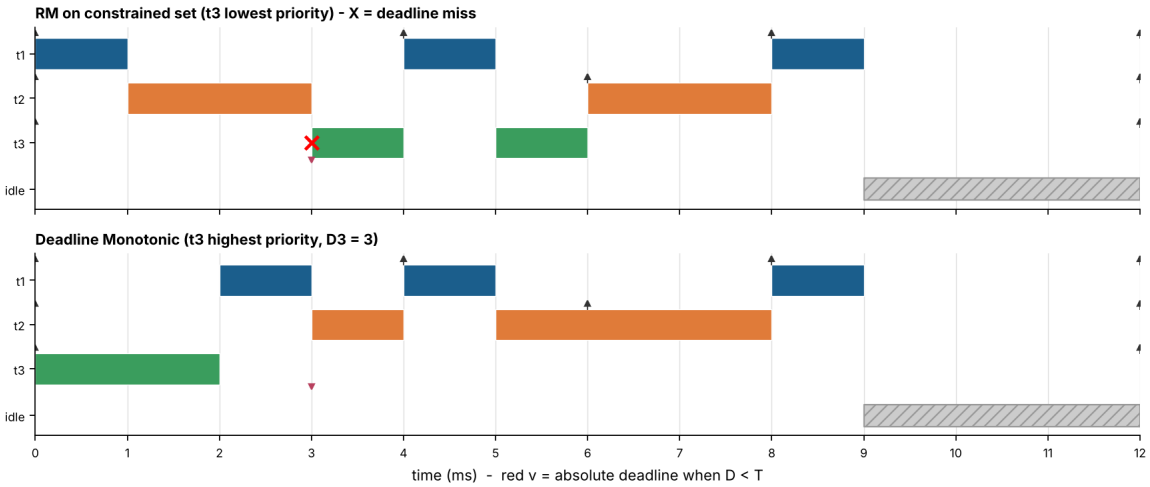


Figure 3. RM versus Deadline Monotonic on the constrained-deadline set; the red X is τ3's deadline miss under RM.

Lab task 2 – Bound sweep

Sweeping n = 1…10 gives 1.000, 0.828, 0.780, 0.757, 0.743, 0.735, 0.729, 0.724, 0.721 and 0.718; the curve decreases monotonically towards ln 2 ≈ 0.693, while the EDF bound stays at 1.0 (Figure 4). The gap between the two curves is the utilization that RM's sufficient test may waste.

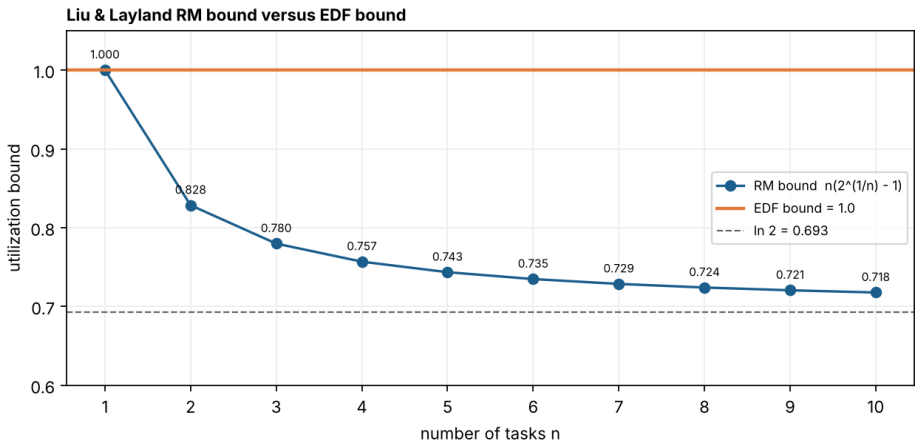


Figure 4. Liu & Layland RM bound versus the constant EDF bound.

Lab task 3 – Least Laxity First

simulate_llf selects the job with the smallest laxity d − t − remaining. On the light four-task set all jobs are 1–2 ticks long, so LLF has no room to thrash and produced the same 15 context switches as RM and

EDF. On a heavier set with longer jobs, a (4, 10), b (5, 12), c (2, 15), U = 0.95, the difference is clear over one hyperperiod (60 ms): RM misses 3 deadlines with 19 context switches, EDF meets every deadline with 15, and LLF also meets every deadline but needs **31 context switches (17 preemptions)**. Because the running job's laxity stays constant while the waiting jobs' laxity drops every tick, two jobs with similar laxity keep overtaking each other – the thrashing visible in the a/b alternation of Figure 5.

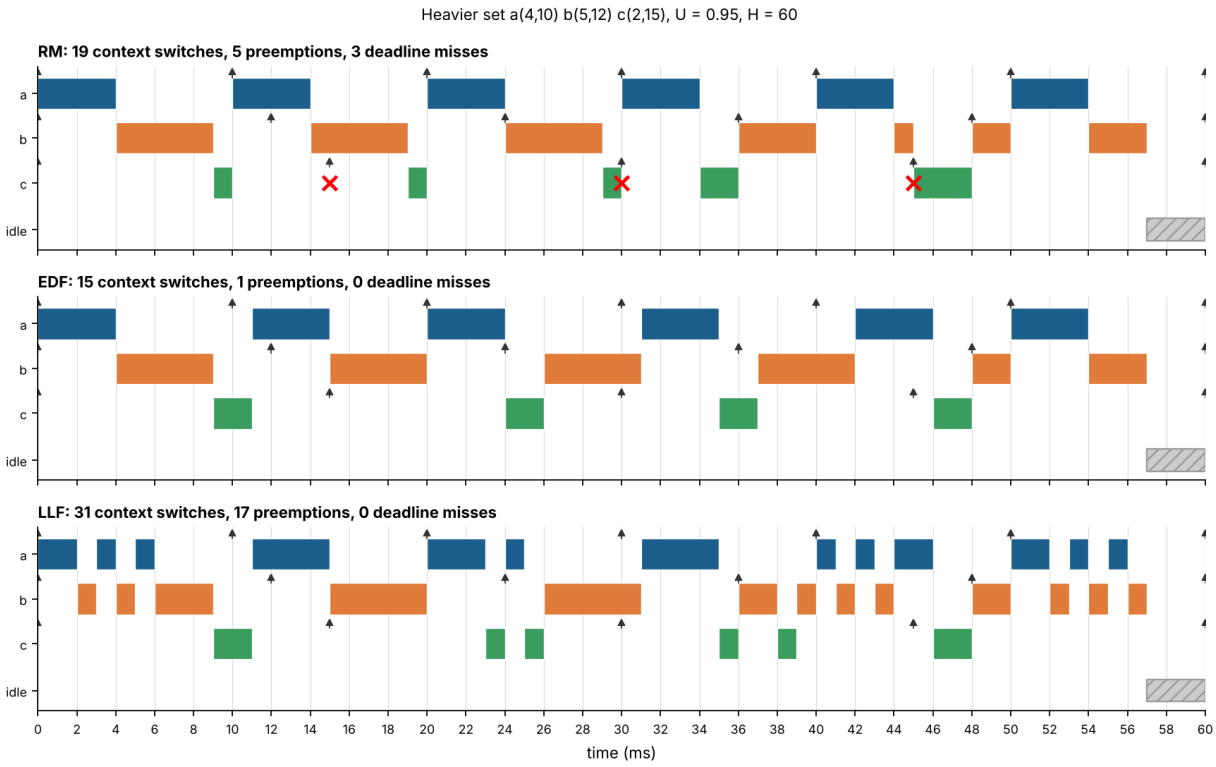


Figure 5. RM, EDF and LLF on the heavier set (U = 0.95) over one hyperperiod.

Lab task 4 – A second medium-priority task

Adding M2 (priority equal to M, arrival t = 3, 3 ticks) during M's execution increases H's unprotected blocking from 9 to 12 ticks, while the PIP-protected blocking stays at 5 ticks (Table 5 and Figure 6).

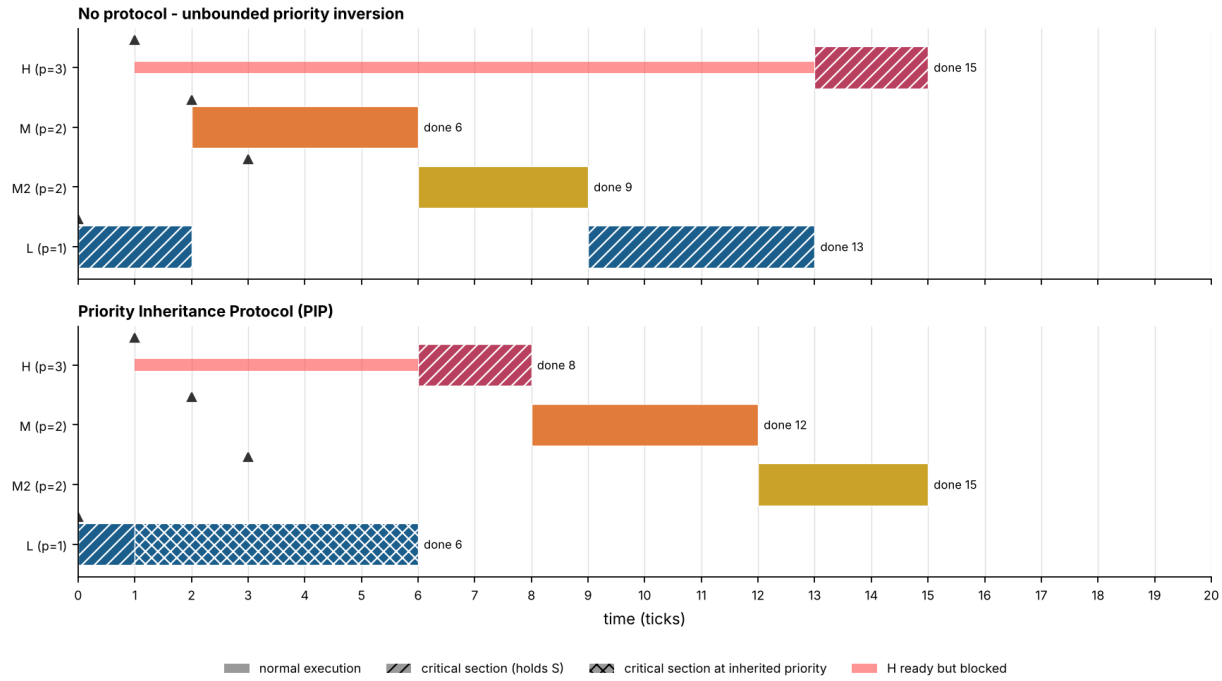


Figure 6. *Inversion with two medium-priority tasks: unprotected blocking grows, PIP blocking does not.*

3. Mini Project 2.2 – RM versus EDF Schedulability Comparison Tool

3.1 Tool design

schedcheck.py reuses the simulators of rts_sim.py and adds an analysis layer. The tool accepts either a Python list of Task objects (analyze(tasks)) or one or more CSV files with columns name, C, T and an optional D (blank D defaults to T). Its decision layer follows Section 2.5.2:

1. Compute u_i, U and the hyperperiod (LCM of all T_i).
2. If every $D_i = T_i$, use RM priorities and test $U \le n(2^{(1/n)} - 1)$. If any $D_i < T_i$, switch to Deadline Monotonic priorities and apply the same bound to the density $\Sigma C_i/D_i$, which is a valid sufficient test for DM.
3. If the bound passes, report schedulable directly; if it fails, run RTA for every task in priority order and report R_i, the iteration count and a per-task verdict.
4. EDF verdict: $U \le 1$ (exact for implicit deadlines). For constrained deadlines $U \le 1$ is only necessary, so the tool additionally runs the exact processor-demand test (demand bound function $\le L$ at every absolute deadline up to H + max D).
5. Cross-check both verdicts by simulating one hyperperiod (capped at 500 time units) and draw a two-panel Gantt chart, RM/DM on top and EDF below, with distinct colours per task, idle time and red crosses at misses.

Robustness: a set with $U > 1$ is reported as infeasible under any algorithm and still charted (the late jobs and misses are visible); RTA stops as soon as $R_i > D_i$, so it cannot loop forever; malformed CSV rows, non-integer values, duplicate names and D > T produce a clear error message and a non-zero exit code instead of a crash.

```
$ python schedcheck.py tasksets/ex_2_3_2.csv
n = 4   U = 0.8750   hyperperiod = 24
Liu & Layland bound n(2^(1/n)-1) = 0.7568   U <= bound ? FAIL
Response Time Analysis (RM priority order):
   Task     prio   R_i   D_i  iter  verdict
   t1          1     1     4     1  OK
   t2          2     3     6     2  OK
   t4          3     4     8     2  OK
   t3          4    11    12     5  OK
RM verdict : SCHEDULABLE (exact RTA)
EDF verdict: SCHEDULABLE (U <= 1)
Simulation over 24 units: RM misses = 0, EDF misses = 0
```

3.2 Results on seven task sets

Table 6. *Summary of the tool's verdicts. Bound test uses density ΣC/D for the constrained set.*

Task set	n	U	L&L bound	Bound test	Fixed priority	EDF	Sim. misses FP / EDF
Section 2.2.2 example	3	0.750	0.780	PASS	RM: OK	OK	0 / 0
Section 2.3.2 example	4	0.875	0.757	FAIL	RM: OK (RTA)	OK	0 / 0
Harmonic, U = 1	4	1.000	0.757	FAIL	RM: OK (RTA)	OK	0 / 0
Eight tasks	8	0.842	0.724	FAIL	RM: OK (RTA)	OK	0 / 0
Constrained deadline	3	0.750	0.780	FAIL	DM: OK (RTA)	OK	0 / 0
RM fails, EDF passes	3	0.950	0.780	FAIL	RM: MISS (RTA)	OK	3 / 0
Overload, U > 1	3	1.250	0.780	FAIL	RM: MISS (RTA)	INFEASIBLE	5 / 5

Table 7. Task-set definitions (files in tasksets/).

Set	Tasks (C, T[, D])
Section 2.2.2 example	t1(1,4), t2(2,6), t3(2,12)
Section 2.3.2 example	t1(1,4), t2(2,6), t3(2,12), t4(1,8)
Harmonic, U = 1	t1(1,4), t2(2,8), t3(4,16), t4(4,16)
Eight tasks	t1(1,10), t2(1,12), t3(2,15), t4(2,20), t5(3,24), t6(3,30), t7(4,40), t8(6,60)
Constrained deadline	t1(1,4), t2(2,6), t3(2,12,3)
RM fails, EDF passes	a(4,10), b(5,12), c(2,15)
Overload, U > 1	t1(2,4), t2(3,6), t3(2,8)

3.3 Written report

We validated the tool on the two worked examples of the unit and on five task sets of our own, chosen so that every branch of the decision layer is exercised. In every case the analytical verdict agreed with the simulated schedule over one hyperperiod (last column of Table 6), which gives us confidence that the utilization tests, the RTA fallback and the simulators are mutually consistent.

Worked examples. The Section 2.2.2 set (U = 0.750) passes the Liu & Layland bound of 0.780, so the tool reports RM-schedulable immediately without running RTA. The Section 2.3.2 set (U = 0.875) fails the bound of 0.757 and the tool falls back to RTA, which returns R = 1, 3, 4 and 11 for τ1, τ2, τ4, τ3 – identical to our hand computation in Section 2.3 – so the set is RM-schedulable. EDF passes in both cases.

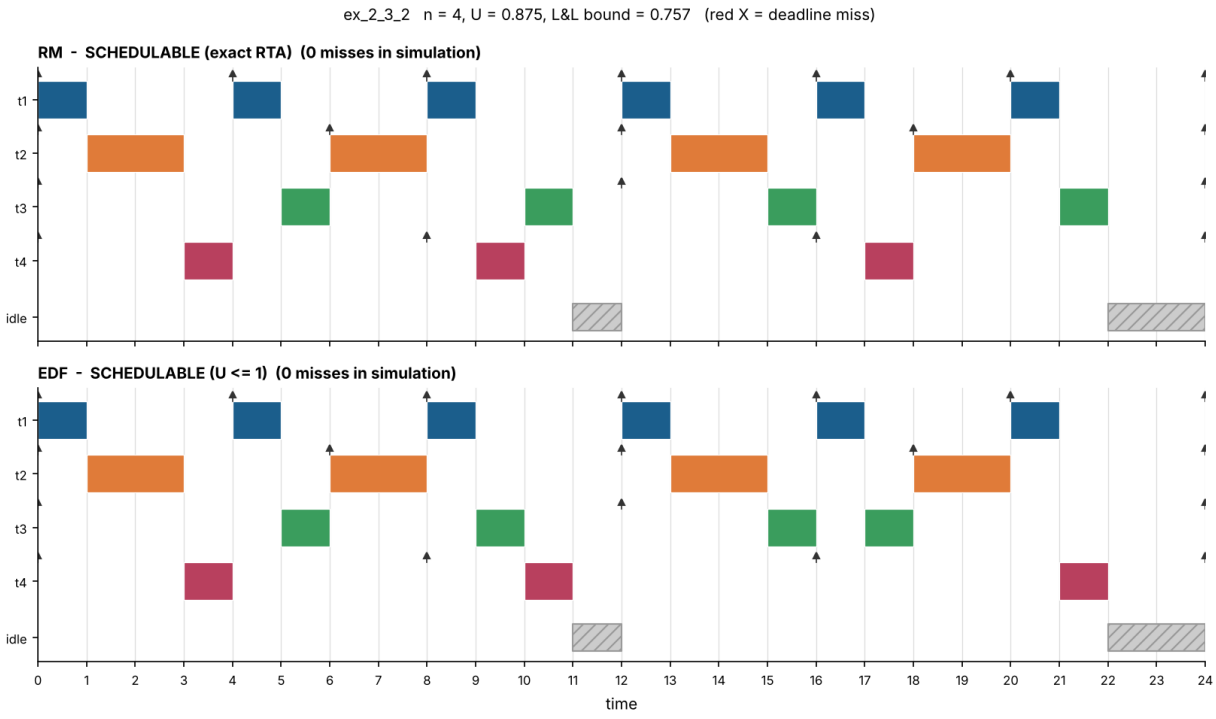


Figure 7. Tool output chart for the Section 2.3.2 set: bound fails, RTA proves RM schedulability; both schedules meet every deadline.

Eight tasks. With periods 10…60 ms and U = 0.842, the bound for n = 8 has already fallen to 0.724, close to its asymptotic ln 2 limit. The sufficient test fails by a wide margin, yet RTA shows that even the

lowest-priority task (C = 6, T = 60) has R = 40 ≤ 60. This illustrates how pessimistic the bound becomes for larger task sets: more than 11 % of the processor would have been declared unusable.

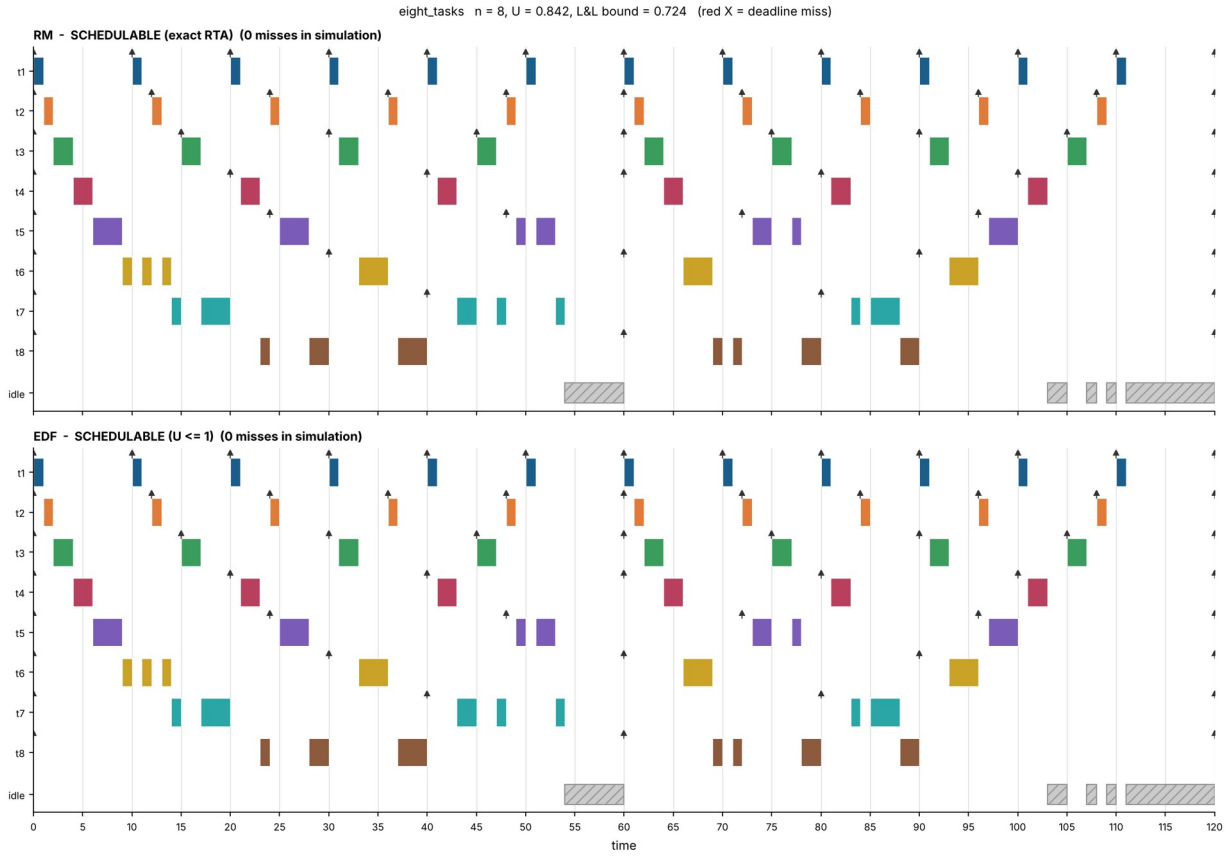


Figure 8. Eight-task set (U = 0.842) over its 120 ms hyperperiod; RM and EDF both meet all deadlines.

Constrained deadline. For τ1 (1,4,4), τ2 (2,6,6), τ3 (2,12,3) the tool detects $D_i < T_i$ and switches to Deadline Monotonic ordering. The density 1.25 exceeds the bound, but RTA under DM finds R = 2, 3, 6 against deadlines 3, 4, 6, so the set is schedulable. This set also shows why U ≤ 1 is not an exact EDF test once deadlines are constrained: the density is above 1 while U is only 0.75, and the exact processor-demand test is what confirms that EDF meets every deadline.

RM fails, EDF succeeds. For a (4,10), b (5,12), c (2,15) with U = 0.95, RTA proves that task c misses its deadline under RM (its response time exceeds 15 ms) and the simulation shows three misses in 60 ms, while EDF schedules the set with no misses. This is the classic case in which the dynamic-priority algorithm uses the processor more fully than any static assignment.

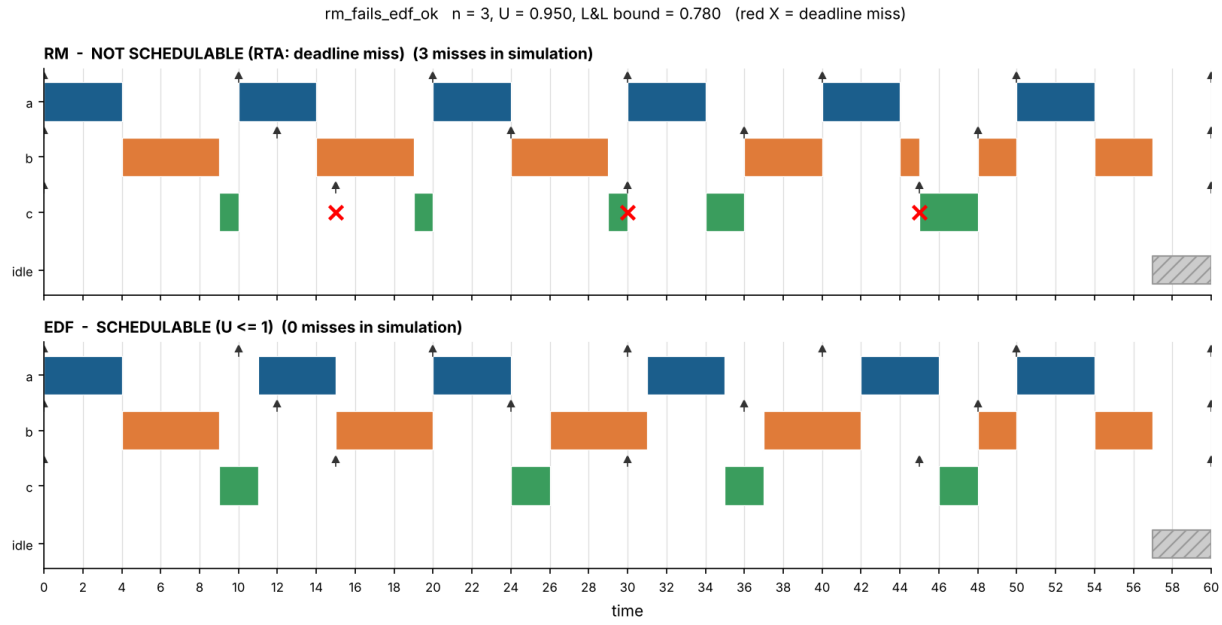


Figure 9. U = 0.95: three deadline misses for task c under RM (red crosses), none under EDF.

Overload. The set (2,4), (3,6), (2,8) has U = 1.25. The tool reports it as infeasible on one processor under any algorithm, RTA marks τ2 and τ3 as missing, and the chart still renders, showing deadline misses under both policies instead of crashing. EDF's misses cascade once the processor is overloaded, which is the well-known domino effect that makes EDF less predictable than RM under overload.

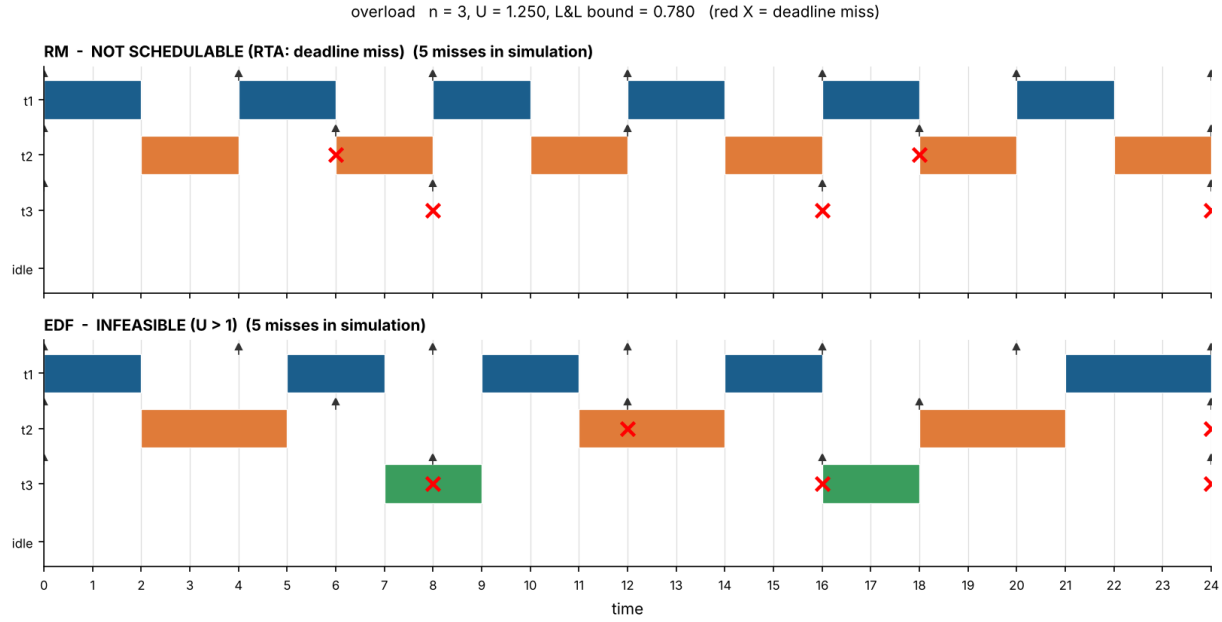


Figure 10. Overloaded set (U = 1.25): the tool reports infeasibility and shows the misses.

Harmonic set at full utilization. Finally, (1,4), (2,8), (4,16), (4,16) has U = 1.0, far above the bound of 0.757, yet RTA gives R = 1, 3, 8, 16 and the simulation is miss-free. When periods are harmonic RM can reach 100 % utilization, so the Liu & Layland bound should never be read as a necessary condition.

3.4 Reflection: when the bound fails but RTA succeeds

The four-task set of Section 2.3.2 is the clearest example of why exact tests matter. Its utilization of 0.875 is above the Liu & Layland bound of 0.757, so a designer who relied only on the bound would

conclude that RM cannot be trusted and might add hardware, lengthen periods or switch to EDF unnecessarily. Response Time Analysis tells a different story: the lowest-priority task τ3 completes after at most 11 ms against a 12 ms deadline, τ4 after 4 ms against 8 ms, and the simulation over the 24 ms hyperperiod confirms these values to the tick. The bound is derived for the worst possible combination of periods, so it is safe but often pessimistic; the more tasks there are, the more capacity it gives away, approaching 31 % as n grows. The harmonic set we built, schedulable by RM at a full 100 % utilization, makes the same point even more strongly. In practice the bound is a fast first filter, but only an exact test such as RTA can certify a set that fails it, and certifying it lets the same processor carry considerably more work.

4. Mini Project 2.3 – Partitioned Multiprocessor Scheduling Simulator

4.1 Design

partition.py turns the m-processor problem into m independent uniprocessor problems plus a bin-packing problem (Section 2.8.1). The "does it fit" predicate is the Liu & Layland bound recomputed for the number of tasks the core would hold after the placement – never a flat 0.69 or 1.0 threshold:

```
core k admits τi  ⇔  U_k + u_i ≤ rm_bound(n_k + 1)    (n_k = tasks on core k)
```

All four heuristics share the signature heuristic(tasks, m) → Partition, which records the task-to-core mapping and any task that could not be placed. Remaining capacity is defined as the slack left under the bound after the placement, rm_bound(n_k + 1) − (U_k + u_i).

Table 8. The four bin-packing heuristics.

Heuristic	Task order	Core chosen among admissible cores
First-Fit (FF)	input order	the first (lowest-index) core
First-Fit Decreasing (FFD)	decreasing u_i	the first core
Best-Fit Decreasing (BFD)	decreasing u_i	the core with the least remaining capacity
Worst-Fit (WF)	input order	the core with the most remaining capacity

4.2 Test sets and comparison table

We used m = 4 and five task sets of increasing difficulty. Periods are drawn from {5, 10, 20, 40} ms so that the per-core hyperperiods stay short enough to simulate and plot.

Table 9. Partitioning test sets (m = 4).

Set	n	Total U	U / m	Distinct u_i values
A balanced	14	2.425	60.6 %	0.15, 0.175, 0.2
B skewed	12	2.650	66.2 %	0.075, 0.1, 0.55, 0.6, 0.65
C tight	10	3.750	93.8 %	0.375
D order sensitive	8	3.200	80.0 %	0.2, 0.6
E adversarial	5	2.750	68.8 %	0.55

*Table 10. Comparison of the heuristics. *Need = cores the heuristic uses when given unlimited cores.*

Set	Heur.	Result	Cores	Per-core U (core 0–3)	Placed U	% of m	Unpl.	Need*	⌈U⌉
A balanced	FF	OK	4/4	0.70 / 0.68 / 0.68 / 0.38	2.425	60.6 %	0	4	3
A balanced	FFD	OK	4/4	0.75 / 0.75 / 0.62 / 0.30	2.425	60.6 %	0	4	3
A balanced	BFD	OK	4/4	0.75 / 0.75 / 0.62 / 0.30	2.425	60.6 %	0	4	3
A balanced	WF	OK	4/4	0.70 / 0.68 / 0.50 / 0.55	2.425	60.6 %	0	14	3
B skewed	FF	OK	4/4	0.75 / 0.78 / 0.75 / 0.38	2.650	66.2 %	0	4	3
B skewed	FFD	OK	4/4	0.75 / 0.78 / 0.75 / 0.38	2.650	66.2 %	0	4	3
B skewed	BFD	OK	4/4	0.75 / 0.78 / 0.75 / 0.38	2.650	66.2 %	0	4	3
B skewed	WF	FAIL	4/4	0.38 / 0.40 / 0.55 / 0.67	2.000	50.0 %	1	12	3
C tight	FF	FAIL	4/4	0.75 / 0.75 / 0.75 / 0.75	3.000	75.0 %	2	5	4
C tight	FFD	FAIL	4/4	0.75 / 0.75 / 0.75 / 0.75	3.000	75.0 %	2	5	4

Set	Heur.	Result	Cores	Per-core U (core 0–3)	Placed U	% of m	Unpl.	Need*	[U]
C tight	BFD	FAIL	4/4	0.75 / 0.75 / 0.75 / 0.75	3.000	75.0 %	2	5	4
C tight	WF	FAIL	4/4	0.75 / 0.75 / 0.75 / 0.75	3.000	75.0 %	2	10	4
D order sensitive	FF	FAIL	4/4	0.60 / 0.80 / 0.60 / 0.60	2.600	65.0 %	1	5	4
D order sensitive	FFD	OK	4/4	0.80 / 0.80 / 0.80 / 0.80	3.200	80.0 %	0	4	4
D order sensitive	BFD	OK	4/4	0.80 / 0.80 / 0.80 / 0.80	3.200	80.0 %	0	4	4
D order sensitive	WF	OK	4/4	0.80 / 0.80 / 0.80 / 0.80	3.200	80.0 %	0	8	4
E adversarial	FF	FAIL	4/4	0.55 / 0.55 / 0.55 / 0.55	2.200	55.0 %	1	5	3
E adversarial	FFD	FAIL	4/4	0.55 / 0.55 / 0.55 / 0.55	2.200	55.0 %	1	5	3
E adversarial	BFD	FAIL	4/4	0.55 / 0.55 / 0.55 / 0.55	2.200	55.0 %	1	5	3
E adversarial	WF	FAIL	4/4	0.55 / 0.55 / 0.55 / 0.55	2.200	55.0 %	1	5	3

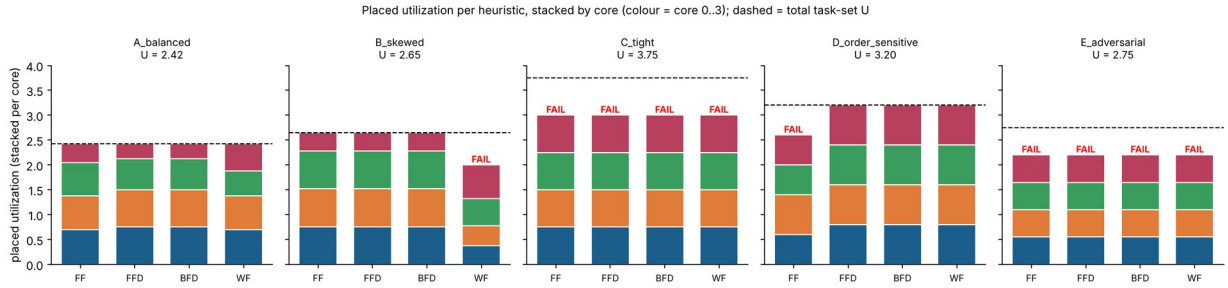


Figure 11. *Placed utilization per heuristic, stacked by core; the dashed line is the task set's total utilization, so a bar below it means some tasks were not placed.*

4.3 Per-core schedule

For the FFD partition of set A we ran the uniprocessor RM simulator independently on each core over one hyperperiod (40 ms). Every core meets every deadline (Figure 12); the largest worst-case response time, 28 ms for a10 (T = 40 ms) on core 1, is still well inside its deadline. Each core is loaded to at most 0.75, below its own bound (0.757 for four tasks), as the admission test guarantees.

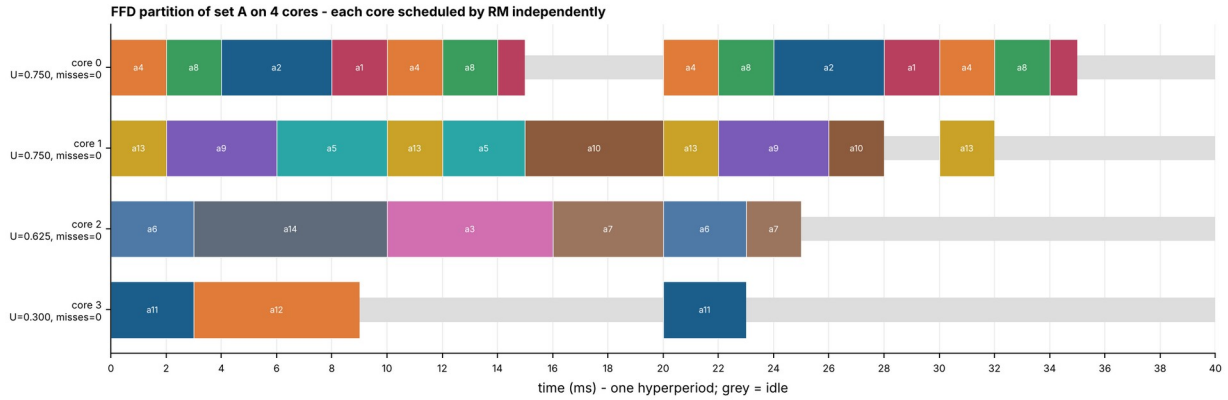


Figure 12. *Per-core RM schedules of the FFD partition of set A; each row is one core, grey is idle time; no deadline is missed.*

4.4 Written analysis

Section 2.8.1 reduces partitioned scheduling to m uniprocessor problems plus an NP-hard bin-packing problem, and Section 2.8.3 notes that practical systems therefore use heuristics with a per-core schedulability test as the fit predicate. Our results show where this works and where it breaks.

Why FFD outperforms FF. Sorting by decreasing utilization places the hardest items first, while every core is still empty and has its full capacity. Set D makes the effect visible: with four small tasks (u = 0.2) listed before four large ones (u = 0.6), FF stacks three small tasks on core 0 (load 0.6), so no large task can join it any more (0.6 + 0.6 > rm_bound(4)). The last large task then finds no admissible core and FF fails, placing only 65 % of m. FFD places one large task per core first and then adds one small task to each, reaching exactly 0.8 per core (just under the two-task bound of 0.828) and succeeding with the optimal four cores. On sets A and B, where the input order happened to be favourable, FF and FFD both succeed, but FFD packs cores 0 and 1 to 0.75 and leaves core 3 almost empty, i.e. it consolidates load and keeps a whole core free for future tasks. BFD behaved like FFD on all our sets.

Why WF is weaker. Worst-Fit spreads load evenly, which looks attractive (set A ends with balanced loads 0.70, 0.68, 0.50, 0.55), but it destroys the large free gaps that big tasks need. In set B the small tasks are spread over all four cores before the u = 0.65 task arrives; by then every core already carries enough load that adding 0.65 would violate its bound, so WF fails while FF, FFD and BFD succeed. With an unlimited supply of cores WF always opens a new, empty core, so it would use one core per task (14 cores for set A).

The adversarial set. Set E contains five tasks of u = 0.55 each. Its total utilization is 2.75, only 69 % of the four-core capacity, yet every heuristic fails. The reason is purely the bin-packing limitation of Section 2.8.1: any two of these tasks together need 1.10, which exceeds the two-task bound of 0.828 and even exceeds a single processor's absolute capacity of 1.0. Each core can therefore hold only one task, five tasks need five cores, and no ordering or choice rule can change this. By hand: $\lceil 2.75 \rceil = 3$ cores would suffice if work were divisible, but partitioning cannot split tasks. Set C makes the same point with smaller items: ten tasks of u = 0.375 (U = 3.75, 94 % of m) fit at most two per core (three would need 1.125), so only eight are placed and 25 % of the platform is unusable.

Distance from the optimum. The trivial lower bound $\lceil U \rceil$ is optimistic because it assumes a core can be filled to 1.0. Under RM admission no core holding three or more tasks can exceed 0.78, and in set A a core with at most two tasks holds at most 0.4, so three cores can carry at most about 2.34 < 2.425. Four cores are therefore genuinely needed, and FFD/BFD's result is optimal under this admission test; in set D FFD also meets the lower bound of four cores exactly. Capacity is lost to the per-core bound itself and to fragmentation; the adversarial sets show that fragmentation can dominate even at low total utilization.

5. Conclusion

The three mini projects reproduced every quantitative result of Unit 2 and extended them. Our simulator confirmed the utilization results of Sections 2.2.2 and 2.3.2 exactly, showed RM and EDF diverging where τ3 and τ4 interleave, and confirmed with RTA that the four-task set is RM-schedulable despite failing the bound. The priority-inversion experiment showed blocking that grows with every medium-priority task (9 → 12 ticks) and PIP holding it at 5 ticks, within the one-critical-section bound. The comparison tool turned these pieces into a reusable analyser that handles implicit and constrained deadlines and overloaded sets gracefully. Finally, the partitioning simulator showed that FFD and BFD outperform FF and WF, and that a set using only 69 % of a four-core platform can still be impossible to partition – the central weakness of partitioned scheduling.

References

[1] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," Journal of the ACM, vol. 20, no. 1, pp. 46–61, 1973.

[2] M. Joseph and P. Pandya, "Finding response times in a real-time system," The Computer Journal, vol. 29, no. 5, pp. 390–395, 1986.

[3] L. Sha, R. Rajkumar and J. P. Lehoczky, "Priority inheritance protocols: an approach to real-time synchronization," IEEE Transactions on Computers, vol. 39, no. 9, pp. 1175–1185, 1990.

[4] J. Y.-T. Leung and J. Whitehead, "On the complexity of fixed-priority scheduling of periodic, real-time tasks," Performance Evaluation, vol. 2, no. 4, pp. 237–250, 1982.

[5] S. K. Dhall and C. L. Liu, "On a real-time scheduling problem," Operations Research, vol. 26, no. 1, pp. 127–140, 1978.

[6] G. C. Buttazzo, Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications, 3rd ed. Springer, 2011.

[7] R. I. Davis and A. Burns, "A survey of hard real-time scheduling for multiprocessor systems," ACM Computing Surveys, vol. 43, no. 4, 2011.

[8] Real-Time Systems, Unit 2 lecture notes and lab manual (Mini Projects), University of Rwanda, 2026.

Appendix A – Reproducing the results

Clone the repository, install matplotlib, and run the scripts from the repository root. Each script prints its tables and writes its charts to figures/ and its text output to results/.

```
git clone https://github.com/hirwagilbert/lab--2-real-time-system-
cd lab--2-real-time-system-
pip install matplotlib
python mp21_run.py                                # Mini Project 2.1
python schedcheck.py tasksets/*.csv --out figures # Mini Project 2.2
python partition.py                               # Mini Project 2.3
python tests/test_all.py                          # hand-computed checks
```

Using the tool from Python with an in-script list:

```
from rts_sim import Task
from schedcheck import analyze, format_report
rep = analyze([Task("t1", 1, 4), Task("t2", 2, 6), Task("t3", 2, 12, D=3)])
print(format_report(rep))
```

Source code and results

All code, task sets, charts and this report are available in the group's GitHub repository:

https://github.com/hirwagilbert/lab--2-real-time-system-